

FastMatch

APK Comparison & Full Testing Report

Static Code Analysis + Live Device Testing

May 2026 · Confidential

Executive Summary

The developers submitted a new APK build claiming all issues had been resolved. This report presents the findings from both a full static code analysis of the new APK and live device testing. The conclusion is clear: the developers have not resolved the critical or medium-severity issues identified in the previous report. Only peripheral infrastructure work (notification permissions, billing SDK integration) was added.

2 Critical Issues Not fixed	14 Runtime Bugs Found in testing	10 Code Issues Still unresolved	3 Items Fixed New APK only
--	---	--	---

Section 1 — Build Comparison: Old vs New APK

Property	Old APK	New APK
File Size	107.4 MB	110.4 MB (+3 MB)
JS Bundle	4.27 MB	4.51 MB (+5.5%)
DEX Files	2 (classes 1-2)	3 (classes 1-3)
Build Tool	AGP 8.6.0	AGP 8.6.0 (same)
Kotlin Version	1.9.24	1.9.24 (same)
Min SDK	API 15 (2011)	API 15 (unchanged)
POST_NOTIFICATIONS	Missing	Added ✓
VIBRATE permission	Missing	Added ✓
Play Billing SDK	Not present	Integrated ✓
Hardcoded IPs	PRESENT	STILL PRESENT ✗
SMS permission code	PRESENT	STILL PRESENT ✗
usesCleartextTraffic	true	true (unchanged) ✗
Code obfuscation	Off	Still off ✗

Section 2 — Live Device Testing: Bugs & Issues Found

The following issues were discovered through direct hands-on testing of the app on a real device. These are functional bugs and UX failures that affect real users right now.

2.1 — Matching System Failures

BUG	Core Matching Still Broken
	The fundamental matching feature does not work reliably. Users are not being matched correctly in multiple scenarios. This is the most important feature in the entire app and remains non-functional.
Fix	Developer must diagnose and fix the core matching engine before any other work continues.

BUG	Skipped / Rejected Profiles Keep Reappearing
	When a user skips or rejects a profile match request, that same profile continues to appear again and again in the matching and request flow. The app is not saving the skip decision. This creates a frustrating and broken experience — users feel they have no control.
Fix	Store skip/reject state persistently in the database per user pair. Never show a rejected profile again unless the user explicitly resets their preferences.

BUG	Declined Call Keeps Reappearing
	Even after a user explicitly declines an incoming call, the same call notification or request continues to appear again. The decline action is not being respected or persisted.
Fix	A declined call must be marked as such on both client and server. Implement backend state to prevent re-triggering declined calls.

BUG	Interest Matching Completely Ignored
	Users with entirely unrelated interests appear as match suggestions. The interest filters users set in their profiles have no visible effect on who they are matched with. Either the matching algorithm ignores interests completely or the filtering logic is broken.
Fix	Add server-side validation and filtering for interests before returning match candidates. Improve the interest-matching scoring algorithm to weight shared interests heavily.

MISSING	No Warning When Connecting With Low-Rated Profiles
---------	--

	The rating system exists in the app, but users receive no alert or warning before being connected with a low-rated profile. Users have no way to make an informed choice about who they are connecting with.
Fix	Before a call connects, show a brief indicator of the matched user's rating. For very low-rated profiles, show an explicit warning and let the user choose to proceed or skip.

MISSING	No Clarity on Match Type — Public, Random, or Private
	Users cannot tell whether they are entering a public random match, a private session, or a filtered interest-based room. The lack of clarity confuses users and damages trust in the experience.
Fix	Clearly label the type of matching session before it begins. Use distinct UI states and descriptive copy for each mode.

MISSING	No Fancy or Interactive Options During a Call
	There are no engaging in-call features to make conversations fun. No AR filters, no interactive games, no reactions, no fun effects. Competitors offer these as standard. This makes calls feel bare and uninteresting.
Fix	Add at minimum: animated emoji reactions, background filters, and a fun "icebreaker" prompt button. These directly improve the quality of matches and time spent in-app.

2.2 — Blocking System: Critically Broken

The blocking feature is fundamentally incomplete. It only removes visibility in some parts of the app but does not terminate the ability to interact. This is a serious safety vulnerability.

CRITICAL	Blocked Users Can Still Edit Messages — Edits Appear for Blocked User
	After User A blocks User B, User B can still edit their old messages in the conversation. These edited messages are visible to User A even though they have blocked User B. This is a direct harassment and spam risk. A blocked user should have zero ability to affect what the blocking user sees.
Fix	Apply hard backend filtering for all interactions from blocked users. Any content action (edit, delete, react) from a blocked user must be invisible to the user who blocked them. Invalidate all interaction permissions immediately on block event.

CRITICAL	Blocked Users Can Still Send Call Requests
	Even after being blocked, a user can still send incoming call requests to the person who blocked them. The block does not prevent call initiation, defeating the entire purpose of blocking for safety.

Fix	Block must apply at the backend level, not just the frontend. The server must reject any call request, message, or interaction from a user ID that has been blocked. There must be no way to bypass this through the API.
-----	---

BUG	No Option to Block a User's Call Requests Specifically
	There is no dedicated option to block call requests from a specific user independently from full blocking. Users may want to stop receiving calls from someone without removing the chat history.
Fix	Add a "Block calls from this user" option in addition to the full block. This gives users finer control over their interactions.

BUG	Blocking Does Not Completely Terminate Interaction Capability
	Overall, the blocking logic is incomplete and unsafe. Blocking a user should completely and immediately end all forms of interaction between the two users — calls, messages, reactions, profile views, and match suggestions. Currently it only partially restricts some of these. The blocking logic is not implemented at the backend level where it must be enforced.
Fix	Blocking must be enforced server-side with a single block check that applies to all interaction types: calls, messages, edits, reactions, and match suggestions. This cannot be handled only in the app's frontend logic.

2.3 — Chat and Messaging Issues

WORKING	Profile Viewing in Chat — Works But Needs Review
	Users can view other users' profile details by tapping the profile icon within a chat. This functionality exists and works. However, it should be verified that blocked users cannot view each other's profiles through this route.
Fix	Verify that the profile view route also respects block status. A blocked user should not be able to view the profile of the user who blocked them.

BUG	Duplicate Notifications — Same Message Triggers Multiple Alerts
	Users receive the same push notification multiple times for a single event, such as a new message or match. This creates notification spam and leads users to disable notifications entirely — destroying a key re-engagement mechanism.
Fix	Deduplicate Firebase and socket event listeners. Use unique, deterministic notification IDs so the same event cannot generate multiple notifications. Audit listener registration to ensure no duplicate subscriptions are created on reconnect.

Section 3 — Code Analysis: Issues Still Not Fixed

These findings were present in the previous APK report and remain completely unaddressed in the new build. None of these were fixed.

Issue	Severity	Status in New APK
Hardcoded internal server IP addresses in JS bundle	Critical	NOT Fixed
SMS / MMS / WAP permission code in bundle	Critical	NOT Fixed
cleartext traffic (HTTP) still allowed	Medium	NOT Fixed
App backup enabled — data copyable via USB	Medium	NOT Fixed
No Network Security Configuration file	Medium	NOT Fixed
android.permission.DUMP in manifest	Medium	NOT Fixed
No certificate pinning for API connections	Medium	NOT Fixed
No code obfuscation (R8 / ProGuard)	Low	NOT Fixed
Minimum SDK still API 15 (Android 4.0, year 2011)	Low	NOT Fixed
"you are logout succesfully" typo in user-facing text	Low	NOT Fixed
Deprecated react-native-incall-manager library	Low	NOT Fixed

Section 4 — New Issues Introduced in This Build

The following problems were not present in the previous APK and have been introduced by the new build.

NEW	READ_CONTACTS and WRITE_CONTACTS Added
	The new build contains references to android.permission.READ_CONTACTS and android.permission.WRITE_CONTACTS. These were absent in the previous version. A video matchmaking app has no legitimate need for write access to the device address book. This needs clear justification from the developer or must be removed.
Fix	Developer must identify which code path or library added these permission references and either justify them fully or remove them.

NEW	ACCESS_NOTIFICATION_POLICY Added
	The new build requests access to the device's Do Not Disturb policy settings. This is unusual for a consumer social app and was not present before. It may have been pulled in by a notifications library but should be reviewed.
Fix	Verify whether this permission is actually needed. If it was added by a third-party library, check whether it can be removed via manifest merging rules.

Section 5 — Developer-Level Technical Recommendations

The following are specific, actionable technical tasks that the developer must implement to bring the app to an acceptable standard. These are not suggestions — they are requirements for a functional, safe, and production-ready application.

5.1 — Backend & Data Layer

- Apply hard backend filtering for blocked users. Blocking must be enforced at the API/server level, not just in the frontend app.
- Store skip/reject state persistently in the database, keyed per user pair, so a rejected profile never reappears.
- Invalidate all interaction permissions immediately after a block event. Calls, messages, edits, and reactions must all be blocked server-side simultaneously.
- Prevent edit visibility of messages after a block event. A blocked user's message edits must not propagate to the blocking user's device.
- Add server-side validation for all matching logic. The matching algorithm must enforce filters (gender, location, interests, language) on the backend, not just the client.

- Improve the interest-matching scoring algorithm to weight shared interests meaningfully in match candidate ranking.

5.2 — Notifications & Real-Time Events

- Deduplicate Firebase and socket event listeners. Audit all listener registration points to ensure no duplicate subscriptions are created, especially on reconnect or app resume.
- Use unique, deterministic notification IDs. Each unique event (message, match, call) must generate exactly one notification — never multiple.
- Implement proper call state management to prevent declined calls from re-appearing. Call state must sync between server and all client devices in real time.

5.3 — State Management & Architecture

- Improve state management consistency across the app. Multiple areas show inconsistency between local state and server state — this is the root cause of several bugs.
- Use proper loading, error, and empty states throughout every screen. No screen should ever appear blank or frozen without an appropriate UI state.
- Implement proper API retry handling with exponential backoff and user feedback for failed requests.
- Implement analytics for failed premium purchase clicks so conversion failure points can be identified and fixed.

5.4 — Quality Assurance & Performance

- Test all flows with real-time multi-device sessions. Many of these bugs only appear when two real devices interact simultaneously.
- Perform memory leak profiling, particularly around call sessions, WebRTC connections, and screen share. Leaks will degrade performance over time.
- Remove hardcoded server addresses from the JS bundle before any release.
- Enable R8 code obfuscation and shrinking in the release build configuration.
- Disable cleartext traffic and add a proper Network Security Configuration file.

Section 6 — Master Issue Checklist

Complete list of all outstanding issues. None of these should be closed until verified by independent testing.

#	Issue	Type	Priority
1	Blocked users can edit messages that appear to blocking user	Runtime Bug	Critical
2	Blocked users can still send call requests	Runtime Bug	Critical
3	Hardcoded internal server IP addresses in JS bundle	Code Security	Critical
4	SMS / MMS permission code present in bundle	Code Security	Critical
5	Core matching not working reliably	Runtime Bug	High
6	Skipped / rejected profiles reappear repeatedly	Runtime Bug	High
7	Declined calls reappear after being dismissed	Runtime Bug	High
8	Blocking does not fully terminate all interactions	Runtime Bug	High
9	Interests have no effect on match suggestions	Runtime Bug	High
10	Duplicate notifications for same event	Runtime Bug	High
11	No warning shown before connecting with low-rated profile	Missing	Medium
12	No clarity on match type (public / random / private)	Missing	Medium
13	No engaging in-call options (filters, reactions, effects)	Missing	Medium
14	No dedicated option to block call requests specifically	Missing	Medium
15	Cleartext HTTP traffic still allowed in app config	Code Security	Medium
16	App data backup enabled — session data copyable via USB	Code Security	Medium
17	No Network Security Configuration file	Code Security	Medium
18	android.permission.DUMP in manifest	Code Security	Medium
19	No certificate pinning for server connections	Code Security	Medium
20	New: READ/WRITE_CONTACTS permissions introduced without justification	New Issue	Medium
21	New: ACCESS_NOTIFICATION_POLICY permission added	New Issue	Low
22	No code obfuscation (R8 / ProGuard)	Code Quality	Low
23	Minimum SDK API 15 — Android 4.0 from 2011	Code Quality	Low

24	"logout succesfully" typo in user-visible text	Code Quality	Low
25	Deprecated incall-manager library still in use	Code Quality	Low